This article has been accepted for publication in IEEE Transactions on Neural Networks and Learning Systems. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TNNLS.2026.3729623

MAB-PGC: Multi-Agent Mutual Awareness Belief via Progressive Graph Construction

***Abstract*—In Multi-Agent Reinforcement Learning (MARL), agents engage in collaborative or competitive interactions, adapting their policies to maximize rewards. However, in environments with only team rewards, agents cannot accurately perceive the rewards obtained from their individual actions. To address this limitation, we propose a multi-agent belief state computation method termed *Mutual Awareness Belief* (MAB). MAB extends belief modeling to decentralized partially observable settings by incorporating agents' behavioral intentions and collaborative relationships. This allows each agent to infer the contribution of individual behaviors to team rewards during decision making and supports team-advantageous joint decisions. Additionally, to address the issue of information lag, we develop Progressive Graph Construction (PGC), a novel module for computing MAB. PGC constructs a dynamic graph with progressively updated node features and cooperation-informed edge weights. Through interaction with PGC, agents iteratively update and retrieve graph information to derive the MAB. Empirical results show that our method achieves competitive performance against baseline approaches on the StarCraft unit micromanagement benchmark and proves effective in the non-monotonic Pursuit environment.**

***Index Terms*—multi-agent reinforcement learning, Mutual Awareness Belief, Progressive Graph Construction.**

I. Introduction

In recent years, Multi-Agent Reinforcement Learning (MARL) [1], [2] based on the Markov Decision Process (MDP) has demonstrated significant success in solving various real-world problems. It has achieved notable advances in fields such as autonomous driving [3], multi-UAV collaboration [4], [5], and game AI [6]. In MARL, multiple agents learn policies concurrently within the same environment [7], with the dynamics of the environment influenced collectively by all participating agents. Early research in MARL often applied single-agent reinforcement learning algorithms directly to multi-agent scenarios [8], with each agent learning independently. However, this approach frequently encountered challenges associated with non-stationarity, especially in decentralized partially observable MDPs (Dec-POMDPs) [9], which capture more realistic multi-agent settings. Non-stationarity arises because the policy of any individual agent evolves continuously due to the policies of other agents.

To tackle these issues, the Centralized Training and Decentralized Execution (CTDE) paradigm has been widely used in the MARL [10]. Nevertheless, traditional centralized training methods often struggle to identify globally optimal decision-making policies. Only using team rewards makes it difficult to precisely attribute contributions to individual agents [11]. As a result, some agents may become less motivated to learn actively and cooperate effectively.

To overcome this limitation, researchers have explored methods to decompose team rewards into individual rewards to allow agents to adjust their policies more efficiently. For example, the Counterfactual Multi-Agent Policy Gradients (COMA) [12] utilizes a counterfactual baseline to estimate individual rewards for each agent. Similarly, value decomposition methods, such as VDN [13], QMix [14], and QTRAN [15], utilize the Individual-Global-Max (IGM) condition to facilitate team reward decomposition by learning individual rewards. However, these methods often fail to explicitly account for the actions of other agents during decision-making [16]. Instead, they treat other agents as part of the environment [17], which limits the ability of agents to accurately assess how their actions affect team rewards. This shortcoming prevents agents from making decisions to maximize the overall performance of the team [18].

In human teams, members usually communicate before making significant decisions, allowing them to assess the potential value of their decisions and collaboratively achieve an effective joint group decision. Similarly, in MARL, communication mechanisms based on collaborative relationships are key to synchronizing information among agents. Some existing methods based on graph networks [19], such as Graph Two Attention neural Network (G2ANet) [20], and Transformer-based Graph Coarsening Network (TGCNet) [21], typically reconstruct the communication topology from scratch at every time step. However, they fail to explicitly preserve historical collaborative relationships. These memoryless approaches to graph construction lead to severe fragmentation of collaboration, causing frequent fluctuations in cooperative relationships and ultimately hindering long-term coordination.

Furthermore, unlike humans, who can instantly share and adapt to new information when changes occur, existing communication methods in MARL lack this capability. In the multi-agent system, the agents' decisions are asynchronous, and actions cannot be performed simultaneously [22]. Hence, the sequence of decisions is randomized and the communicated information is unable to be changed by the action of another agent, leading to a misalignment between the information and the actual state of the agent who makes a decision later. This issue, known as "information lag", often leads agents to make decisions based on outdated information, which can mislead the entire decision process. To address this issue, the bidirectional action-dependent Q-learning (a.k.a. ACE) aims to replace the uncertain order of decision-making in MARL with a fixed order, enabling the information to be passed in sequence. In ACE, the first agent generates a message, which subsequent agents can modify or augment as

needed [18]. However, the ACE still has some drawbacks: (1) information is conveyed through successive estimated state transition functions, leading to potential message distortion; (2) fixed-order decisions will block the decision-making process of the agent, and (3) the use of the global state is inconsistent with the real-world situation.

To overcome these challenges and enable agents to better assess the potential value of their own decisions, we propose the *Mutual Awareness Belief* (MAB) method. This method extends the belief concept in MDPs to Dec-POMDPs by incorporating real-time behavioral characteristics (e.g., up-to-date state and intentional state) of each agent. Through MAB, agents can learn about the potential actions of other agents during the decision-making process, resulting in a more accurate assessment of each agent's contribution to the team while maximizing the rewards through the team's joint actions. Furthermore, to better represent MAB and overcome the information lag issue, we propose a novel *Progressive Graph Construction* (PGC) module. The PGC employs a weakly centralized information center that interacts with each agent's information, while preserving decentralized execution in decision-making. The PGC consists of two important components: (i) an agent behavior update component for representing the observation transition in the MAB, and (ii) a collaboration relationship update component for representing the state transition in the MAB. Both components construct transition spaces independently and iteratively update these spaces based on agents' real behaviors. This update mechanism explicitly preserves historical collaborative context to smooth the transition of cooperation, thereby effectively avoiding the collaborative fragmentation caused by memoryless graph construction. As such, embedding real behaviors into the update of transition spaces effectively mitigates the message distortion problem and enables robust long-term coordination. The main contributions of this paper are summarized as follows:

1) MAB is proposed by expanding the belief state to the joint observation and action space to help agents more accurately assess the contribution of actions to the team.
2) PGC module is designed to construct the observation transition function and state transition function more effectively based on joint observation and joint action, addressing the computational complexity of MAB.
3) The proposed MAB-PGC is evaluated on multiple maps in the StarCraft micro-management environment and the Pursuit environment. The results demonstrate the effectiveness of our method compared with the baselines.

II. Background and Related Work

A. Problem Formulation

Decentralized partially observable Markov decision process (Dec-POMDP) [23] is defined as a multi-agent sequential decision-making task, which is represented as a tuple: $< N, \mathcal{S}, \boldsymbol{\mathcal{A}}, \boldsymbol{\mathcal{O}}, P, \mathcal{R}, \Omega, \gamma >$, where N is the number of agents; $\mathcal{S}$ denotes the space of environmental states. $\boldsymbol{\mathcal{A}} = \{\mathcal{A}_i\}_{i=1,2,...,N}$ represents the action spaces for all agents, where $\mathcal{A}_i$ is the action set of agent i. $\boldsymbol{\mathcal{O}} = \{\mathcal{O}_i\}_{i=1,2,...,N}$ represents the observation spaces for all agents, where $\mathcal{O}_i$ is the observation set of agent i. $P(s'|s, \mathbf{a}) : \mathcal{S} \times \boldsymbol{\mathcal{A}} \times \mathcal{S} \to [0, 1]$ represents the environmental state transition function, defining the probability of transitioning to state s' given the current state s and the joint action $\mathbf{a} = (a_1, a_2, ..., a_N)$. The team reward function is $\mathcal{R} : \mathcal{S} \times \boldsymbol{\mathcal{A}} \to \mathbb{R}$, which maps a state and joint action to a shared reward. The observation function $\Omega(s, i) : \mathcal{S} \to O_i$ determines the private observation $o_i^t \in O_i$ for agent i at time t, which is derived as $o_i^t = \Omega(s^t, i)$. Finally, $\gamma \in [0, 1]$ denotes the discount factor. Each agent aims to learn a policy $\pi_i(a_i|o_i)$ that maximizes the expected discounted global return $\mathbb{E}[\sum_{t=0}^{\infty} \gamma^t R_{i,t}]$, where $R_{i,t} \sim \mathcal{R}(s^t, \mathbf{a}^t)$.

In Dec-POMDP, agent i only has access to the observation o_i^t of the current time step t, $o_i^t \in \mathcal{O}_i$. Based on the information, each agent needs to make a decision (i.e., select an action) at a step. In order to make more information available to agents, observations and actions in previous steps, as well as a priori estimation of the initial state, can be used to build a certain a posteriori estimation of the current true state, referred to as the Belief State [24]. Formally, the belief state of agent i is defined as $b_i^t(s^t|o_i^0, ..., o_i^t, b_i^0(s^0))$, i.e., the probability of an agent currently being in state s^t, conditioned on the initial belief state $b_i^0(s^0)$ and the historical information up to time t.

Based on the Dec-POMDP framework, researchers introduced Multi-Agent Reinforcement Learning (MARL), which incorporates reinforcement learning mechanisms to enable agents to iteratively improve their policies through trial-and-error interactions with the environment and other agents.

B. MARL Method

In the process of trial-and-error, non-stationarity arises because the policy of each agent continuously evolves in response to the changing policies of other agents. To address the challenges posed by non-stationarity in MARL tasks [25], many algorithms have been proposed in recent years. In general, they can be categorized based on the degree of centralization into the independent learning methods [26] and the communication-based learning methods [27].

1) Independent learning methods: Independent learning methods typically involve the critic and policy networks. The critic network learns from all agents' local observations during training, while agents rely only on their own observations to make decisions during execution. The critic network improves the team reward decomposition for training policies of agents. For example, VDN factorizes the Q-values of combined actions by decomposing the sum of individual agent Q-values [13]. QMIX extends VDN by enabling the joint action Q-value to be a monotonic combination of each agent's proxy Q-value [14]. QTRAN further enhances QMIX to tackle non-monotonic decomposition challenges in Q-values [15]. However, while VDN, QMIX, and QTRAN adhere to the IGM conditions to obtain joint Q-values, they do not explicitly consider the reasonableness of a single agent's Q-value. Agents do not explicitly consider the behavior of other agents during the decision-making process, resulting in inaccurate estimates of action values [18]. For instance, if an agent selects an essentially bad action while the team receives a substantial

reward, this team-wise positive feedback is misleading the agent into being inclined to select unfavorable actions when facing similar states. This situation becomes more problematic as the number of agents increases. The misestimation of rewards will be accumulated in each agent's Q-value. In this case, even if the IGM condition is met, the computation of the joint Q-value may still yield inaccurate results.

2) Communication-based learning methods: Communication methods, e.g., CommNet, ACE [20], [28], offer an effective way to supplement an agent's local observations with exchanged information. However, exchanging identical information among all agents during this process prevents them from establishing precise cooperative relationships [21], [29]. To address this, researchers have proposed graph-based MARL communication methods that explicitly model collaborative relationships between agents using graphs. This approach better aggregates communication content and promotes coordination. For instance, G2ANet employs a two-stage attention mechanism to construct a collaboration graph for aggregating agent observations [20]. This process allows agents to exchange only the information considered most beneficial for decision-making. Another state-of-the-art example is TGCNet, which employs a graph coarsening network to approximate the global state from local observations [21], thereby eliminating the need for the true global state, which is often unavailable during centralized training. However, the collaborative graphs in these methods are primarily constructed based on information available at the current time step. This approach can be considered short-sighted because it does not explicitly inherit collaborative relationships learned from previous steps. This leads to a strong sense of fragmentation in agent collaboration: two agents may exhibit strong collaborative relationships in one time step but lack such collaboration in the next, hindering the ability of agents to maintain long-term coordination.

Other researchers contend that the aforementioned methods suffer from information lag. Consequently, they explore sequential communication-based decision-making approaches. For example, ACE transforms multi-agent MDPs [30] into single-agent Markov decision processes (MDPs), termed Sequential Extended MDPs (SE-MDPs) [18], thereby establishing bidirectional dependencies. This approach allows the agent that acts first to transmit state predictions based on its chosen action, informing the next agent of the potential consequences. While this resolves the information lag issue, the continuous relaying of predicted states, where predictions for the next agent are derived from those transmitted by the previous agent, introduces information distortion problems.

Our method, MAB-PGC, differs from existing methods in several ways, although it also enables agents to acquire additional information through communication. Specifically, the graph in MAB-PGC is not rebuilt from scratch at each step, but is instead iteratively updated, allowing it to maintain a persistent and evolving understanding of agent relationships over time. This fundamental difference in design philosophy enables MAB-PGC to capture longer-term dependencies in agent cooperation. This design permits agents to access information from the PGC module at any time, enabling independent decision-making without needing to collect data from all other agents or wait for previous agents to act. Furthermore, the PGC incorporates transition spaces that are updated in real-time with observations after each agent makes a decision. This immediate updating ensures that the method leverages current and accurate information when updating transition spaces, thereby avoiding the information distortion that occurs in ACE.

III. The Method

This section introduces the Mutual Awareness Belief (MAB). Using Bayesian principles, we establish a progressive update theory for MAB to provide a mathematical foundation for multi-agent coordination. We first bound the value estimation errors under belief perturbations (Lemma 1) to establish the equicontinuity of the value functions (Theorem 2). Consequently, the agents' best-response policy mapping is proven to be topologically closed and convex (Lemmas 2 and 3, Theorem 3). Fulfilling the prerequisites of Kakutani's Fixed Point Theorem (Theorem 1), we demonstrate the existence of a Markov Perfect Bayesian Equilibrium (MPBE) within the system (Proposition 1). Furthermore, the convergence of the proposed MAB-Bellman operator is theoretically analyzed (Proposition 2). Exact computation of these posterior probabilities is often intractable in complex environments. To bridge this gap, we design the MAB-PGC architecture. It practically approximates the theoretical model through a continuous *retrieve* and *update* cycle. During the Retrieve phase, an agent queries a graph structured by the observation feature space $\mathbf{B}^o$ and the relationship space $\mathbf{B}^r$. This operation extracts the MAB (Definition 1) to guide local decision-making. In the Update phase, the agent's intent features are broadcast to the PGC module. The nodes and edge weights of graphs are then progressively refined. Ultimately, this mechanism simulates the underlying transition dynamics, ensuring that agents consistently access a dynamically updated global context during decentralized execution.

A. Definition of Mutual Awareness Belief (MAB)

We extend belief states from an agent's individual observations and actions to encompass the observations and actions of multiple agents. The joint observation in the multi-agent system at step t is defined as,

$$\mathbf{o}^t = [o_1^t, o_2^t, \ldots, o_N^t] \tag{1}$$

where o_i^t is the observation of agent i at step t. The joint action of agents at step t is represented as,

$$\mathbf{a}^t = [a_1^t, a_2^t, \ldots, a_N^t] \tag{2}$$

where a_i^t is agent i's action at step t. The joint belief state $B_t(s^t)$ of the multi-agent system is defined as,

$$\begin{aligned} B_t(s^t) &= \tau(B_{t-1}, \mathbf{a}^{t-1}, \mathbf{o}^t) \\ &= \eta P_T(\mathbf{o}^t | \mathbf{a}^{t-1}, s^t) \\ &\quad \sum_{s \in \mathcal{S}} P_T(s^t | s^{t-1}, \mathbf{a}^{t-1}) B_{t-1}(s^{t-1}), \end{aligned} \tag{3}$$

where $\tau(B_{t-1}, \mathbf{a}^{t-1}, \mathbf{o}^t)$ denotes the updated belief state after executing the joint action $\mathbf{a}^{t-1}$ and receiving the joint observation $\mathbf{o}^t$ at the previous joint belief state B_{t-1}. Here η is a normalisation factor that ensures that the $B_t(s^t)$ is a valid probability distribution, $P_T(\mathbf{o}^t|\mathbf{a}^{t-1}, s^t)$ is the observation transition function and $P_T(s^t|s^{t-1}, \mathbf{a}^{t-1})$ is the state transition function. $\eta = \frac{1}{P_T(\mathbf{o}^t|B_{t-1}, \mathbf{a}^{t-1})}$, $P_T(\mathbf{o}^t|B_{t-1}, \mathbf{a}^{t-1})$ as follows,

$$P_T(\mathbf{o}^t|B_{t-1}, \mathbf{a}^{t-1}) = \sum\nolimits_{s^t \in S} P_T(\mathbf{o}^t|\mathbf{a}^{t-1}, s^t) \sum_{s^{t-1} \in S} P_T(s^t|s^{t-1}, \mathbf{a}^{t-1}) B_{t-1}(s^{t-1}). \tag{4}$$

Still, when extended to the joint observations and actions of multiple agents, the combinatorial number of observations and actions to be considered for the two functions increases greatly. The computational difficulty and cost are significantly increased. To address this problem, we convert the computational process of $B_t(s^t)$ to the computation of graph structure, by which $P_T(\mathbf{o}^t|\mathbf{a}^{t-1}, s^t)$ is denoted as the featuralization function of the agent nodes, and $P_T(s^t|s^{t-1}, \mathbf{a}^{t-1})$ is denoted as the transition of relationships between the agent nodes. The cumulative computation is then converted into parallelizable matrix operations over graph structures. Further details on the graph representation of MAB states $B_t(s^t)$ are provided in Section III.B.

On the other hand, the transition function $P_T^i(s^t|s^{t-1}, \mathbf{a}^{t-1})$ for each agent's belief state $b_i(s^t)$ is conditionally independent and differently distributed. Although we use parameter sharing in the practical implementation to improve sample efficiency [31], the shared network is conditioned on each agent's local observation and one-hot Agent ID. Therefore, the same parameter set can represent different agent-conditioned functions [32], [33]. Under different Agent ID conditions, the hidden feature h_i^t and the adjustment term $p_i(\theta_i|h_i^t)$ can vary across agents. Thus, parameter sharing does not force all agents to use identical belief-transition approximations; instead, it provides an agent-conditioned approximation consistent with the conditionally independent formulation used in our MAB definition. In other words, the trust belief distribution of the MAB differs for each agent. Therefore, the joint belief state $B_t(s^t)$ needs to be further processed to ensure that agents' $b_i(s^t)$ conform to the distribution of each agent's conditionally independent transition function $P_T^i(s^t|s^{t-1}, \mathbf{a}^{t-1})$. To this end, the transition function $P_T(s^t|s^{t-1}, \mathbf{a}^{t-1})$ is adjusted using $p_i(\theta_i|h_i^t)$, which is the sampling of the current observation feature h_i^t through the fully connected layer θ_i of the policy network. The adjusted state transition function for each agent is defined as,

$$P_T^i(s^t|s^{t-1}, \mathbf{a}^{t-1}) = P_T(s^t|s^{t-1}, \mathbf{a}^{t-1}) p_i(\theta_i|h_i^t), \tag{5}$$

such that the MAB is adapted to match the state transition function P_T^i of each agent's perspective. The final belief state of each agent $b_i(s^t)$ is then computed as,

$$b_i(s^t) = \eta P_T(\mathbf{o}^t|\mathbf{a}^{t-1}, s^t) \sum_{s^{t-1} \in S} P_T^i(s^t|s^{t-1}, \mathbf{a}^{t-1}) B_{t-1}(s^{t-1}). \tag{6}$$

Since we have,

$$\begin{aligned} b_i(s^t) =& \eta P_T(\mathbf{o}^t|\mathbf{a}^{t-1}, s^t) \sum_{s^{t-1} \in S} P_T^i(s^t|s^{t-1}, \mathbf{a}^{t-1}) B_{t-1}(s^{t-1}) \\ =& \eta P_T(\mathbf{o}^t|\mathbf{a}^{t-1}, s^t) \sum_{s^{t-1} \in S} P_T(s^t|s^{t-1}, \mathbf{a}^{t-1}) p_i(\theta_i|h_i^t) B_{t-1}(s^{t-1}) \\ =& B_t(s^t) p_i(\theta_i|h_i^t), \end{aligned} \tag{7}$$

the belief state $b_i(s^t) = B_t(s^t) p_i(\theta_i|h_i^t)$ for agent i is expressed as the joint belief state $B_t(s^t)$ scaled by the agent-specific observation-dependent term $p_i(\theta_i|h_i^t)$.

Definition 1 (Mutual Awareness Belief, MAB)**.** *In a multi-agent system, the Mutual Awareness Belief (MAB) is a global representation that extends an individual's belief to encompass the joint observation $\mathbf{o}^t$ and joint action $\mathbf{a}^t$. The joint belief state $B_t(s^t)$ at time step t is updated based on the previous state B_{t-1}, alongside the state and observation transition functions, and is normalized by a factor η. Using the agent-conditioned transition approximation, the system scales $B_t(s^t)$ using an agent-specific, observation dependent term $p_i(\theta_i|h_i^t)$. Ultimately, the MAB for a specific agent i is defined as $b_i(s^t) = B_t(s^t) p_i(\theta_i|h_i^t)$, thereby ensuring alignment with the conditionally independent state transition from the individual agent's perspective.*

Based on the MAB, agents can estimate the current team reward at each step as follows,

$$R(\mathbf{a}^t, \{b_i(s^t)\}_{i=1,2,\ldots,N}) = \sum_{i=1}^{N} b_i(s^t) R_{i,t}(s^t, \mathbf{a}^t), \tag{8}$$

where $R_{i,t}(s^t, \mathbf{a}^t)$ is the reward estimate for each agent generated by the MixNet. Accordingly, the agent's policy function π is based on the $b(s^t)$, and is defined as $a \sim \pi(b(s^t))$.

The value function Q corresponding to the policy π for MAB, is expressed as,

$$\begin{aligned} &Q(s^t, \mathbf{a}^t, \{b_i\}_{i=1,2,\ldots,N}) \\ &= \mathbb{E}_{p(\theta|h^i)}[\mathbb{E}[\sum_{t=0}^{\infty} \gamma^t R(\mathbf{a}^t, \{b_i(s^t)\}_{i=1,2,\ldots,N})]], \end{aligned} \tag{9}$$

where γ is the discount factor. The optimal action value function Q_* of the extended MDP with the b_i satisfies the Bellman optimality equation as below,

$$\begin{aligned} &Q_*(s^t, \mathbf{a}^t, \{b_i(s^t)\}_{i=1,2,\ldots,N}) \\ &= \max_{\mathbf{a} \in \mathcal{A}} [R(\mathbf{a}^t, \{b_i(s^t)\}_{i=1,2,\ldots,N}) + \\ &\qquad \gamma \sum P_T(\mathbf{o}^t|B_{t-1}, \mathbf{a}^{t-1}) \\ &\qquad Q_*(s^{t+1}, \mathbf{a}^{t+1}, \{b_i(s^{t+1})\}_{i=1,2,\ldots,N})]. \end{aligned} \tag{10}$$

A joint cooperative policy $\pi_* = (\pi_*^i)_{i \in \mathcal{N}}$ forms an *ex interim* Markov perfect Bayesian equilibrium (MPBE), if for all $p(\theta|h), s \in \mathcal{S}$, have $\pi_* \in arg\ max \mathbb{E}_{\pi(\cdot|H)}[V_\theta(s)]$, where $V_\theta(s)$ is state value function. This framework allows each agent to make the best decision to maximize its expected value

based on the current posterior belief under the current observation. The following hypotheses and propositions support this approach:

Hypothesis 1. *Beliefs and policies are assumed to be updated under the following conditions: (1) Consistency. At each time step t, each agent updates transition spaces in PGC according to Bayes' rule. (2) Sequential rationality. Each policy maximises the expected value function under belief* b_i.

To analyze the stability and convergence of our proposed method, we establish a theoretical foundation centered on the Markov Perfect Bayesian Equilibrium (MPBE). Since MAB fundamentally restructures the agents' perceived state space, it is imperative to prove that policies trained on these beliefs will not diverge, but rather converge to a stable equilibrium. Below, we first prove the existence of this MPBE using Kakutani's fixed-point theorem (Proposition 1), ensuring that a stable collaborative policy mathematically exists. Subsequently, we prove that our specific MAB-Bellman updates converge to the corresponding optimal value under the stated assumptions (Proposition 2).

Proposition 1 (Existence of MPBE)**.** *Assuming a Dec-POMDP with the finite agent, finite states, observations and action spaces, then ex-post mixed-policy Markov perfect Bayesian equilibria exist.*

Proof: This proof aims to establish the existence of the MPBE. The ultimate mathematical conclusion relies on Kakutani's Fixed Point Theorem (subsequently formally stated as Theorem 1), which guarantees the existence of an equilibrium provided that the multi-agent best-response mapping ϕ and its domain (the joint strategy space $\prod_i \Delta(\mathcal{A}^i)$) satisfy strict topological prerequisites, particularly regarding compactness, convexity, and continuity. Theorem 1, which theoretically confirms that at least one optimal equilibrium $\boldsymbol{\pi}^* \in \phi(\boldsymbol{\pi}^*)$ inherently exists. This ensures that Proposition 1 holds. ■

Before presenting the proof, we clearly define $V_\theta(s)$ as follows,

$$V_\pi(s) = \sum_a \pi(\mathbf{a}|b(s^t))Q_\pi(s^t, \mathbf{a}^t, \{b_i\}_{i=1,2,\ldots,N}) \tag{11}$$

Theorem 1 (Kakutani's fixed point theorem) If X is a closed, bounded, and convex set in a Euclidean space, and ϕ is an upper semicontinuous correspondence mapping X into the family of closed, convex subsets of X, then $\exists x \in X,$ s.t. $x \in \phi(x)$.

Proof: To successfully apply Kakutani's Fixed Point Theorem to our specific MARL formulation, we need to show that our joint best-response mapping ϕ strictly satisfies the two specified topological prerequisites. The subsequent theoretical analysis is entirely dedicated to proving these exact MARL properties. First, we introduce Theorem 2 to establish the global stability of the system by proving the equicontinuity of the value function set. The microscopic foundation for this theorem is immediately provided by Lemma 1, which formally establishes local stability via an $\epsilon - \delta$ relationship. By leveraging the inherently bounded nature of environmental rewards ($|R| \leq R_{max}$), Lemma 1 explicitly bounds single-step value variations against additive local perturbations in joint policies, mutual awareness beliefs, and future value estimations. This local guarantee ensures that step-wise errors do not compound divergently. Next, Lemma 2 formally verifies the required convexity of the resulting best-response policy space. Subsequently, Theorem 3 extends our stability guarantees, proving that the established equicontinuity is preserved under the Bayesian expectation operator concerning opponents' policies. Finally, relying on this unbroken chain of stability, Lemma 3 utilizes the closed-graph property to establish the upper hemicontinuity of the mapping ϕ. By sequentially establishing these foundational properties, we satisfy all necessary preconditions, culminating in Theorem 1, which theoretically confirms that at least one optimal equilibrium $\boldsymbol{\pi}^* \in \phi(\boldsymbol{\pi}^*)$ inherently exists. ■

In the context of our specific MARL formulation, the domain X in Kakutani's theorem corresponds to the joint policy space over the action space $\mathcal{A}$ (i.e., $\prod_i \Delta(\mathcal{A}^i)$), which inherently forms a convex simplex. The correspondence ϕ maps a joint policy $\boldsymbol{\pi}$ to its best-response set.

Theorem 2 Assume uncertainties in transition probabilities and payoffs belong to compact sets, all agents use stationary policies, then the set $\{V^i_{\pi,b_i(s)}\}$ is equicontinuous.

Proof: This theorem provides the functional-analytic foundation for the stability of our equilibrium. To prove the equicontinuity of the set $\{V^i_{\pi,b_i(s)}\}$, we utilize Lemma 1 to decompose the difference between two successive value function updates into terms controlled by policy distance, belief distance, and next-state value distance [34]. By establishing that the MAB-Bellman operator is a contraction under the specified clipped loss function (Eq. 17), we demonstrate that the function set is uniformly bounded and equicontinuous. This ensures that small perturbations in agents' beliefs or policies do not lead to divergent value estimations, satisfying the compactness requirements for the existence of an equilibrium. ■

Lemma 1 : (Equicontinuity of $\{V_s(\pi, b_i, V_{s',i})\}$) For any $\epsilon > 0$, there exists a $\delta > 0$ such that if $d_1(\pi_1, \pi_2) + d_2(b_{1,i}, b_{2,i}) + d_3(V^i_{1,s'}, V^i_{2,s'}) < \delta$, then,

$$|V^i_s(\pi_1, b_{1,i}, V^i_{1,s'}) - V^i_s(\pi_2, b_{2,i}, V^i_{2,s'})| < \epsilon, \tag{12}$$

where $d_1(\pi_1, \pi_2) = \max_i |\pi_{1,i} - \pi_{2,i}|$ represents the policy distance, $d_2(b_{1,i}, b_{2,i}) = \max_i |b_{1,i} - b_{2,i}|$ is the belief distance, and $d_3(V^i_{1,s'}, V^i_{2,s'}) = \max_{s'} |V^i_{1,s'} - V^i_{2,s'}|$ is the next-state value function distance. Based on the Bellman equation and the triangle inequality, we can decompose the difference into two main parts.

Proof: Let $\pi_1 = \pi_2 + \Delta\pi$ and $V_{1,s'} = V_{2,s'} + \Delta V_{s'}$. We examine the absolute difference of the value functions,

$$\begin{aligned} &|V^i_s(\pi_1, b_{1,i}, V^i_{1,s'}) - V^i_s(\pi_2, b_{2,i}, V^i_{2,s'})| \\ &= \left| \sum_{a \in A} \left(\prod_{i=1}^{|I|} \pi_{1,s,a^i,i} \right) \left(R_{s,a,i} + \gamma \sum_{s' \in S} P_{s,a,s'} V^i_{1,s'} \right) \right. \end{aligned}$$

$$-\sum_{a\in A}\left(\prod_{i=1}^{|I|}\pi_{2,s,a^i,i}\right)\left(R_{s,a,i}+\gamma\sum_{s'\in S}P_{s,a,s'}V^i_{2,s'}\right)\Bigg| \tag{13}$$

$$\leq \underbrace{\left|\sum_{a\in A}R_{s,a,i}\left(\prod_{i=1}^{|I|}\pi_{1,s,a^i,i}-\prod_{i=1}^{|I|}\pi_{2,s,a^i,i}\right)\right|}_{\text{Part 1: Due to Reward}}$$

$$+\underbrace{\left|\gamma\sum_{a,s'}P_{s,a,s'}\left(\prod_{i=1}^{|I|}\pi_{1,s,a^i,i}V^i_{1,s'}-\prod_{i=1}^{|I|}\pi_{2,s,a^i,i}V^i_{2,s'}\right)\right|}_{\text{Part 2: Due to Future Value}}.$$

Since π is a probability function, we have $|\pi| \leq 1$. Additionally, since the reward is finite and $\gamma \leq 1$, it follows that $|\{V_s(\pi, b_i, V_{s',i})\}|$ is also bounded. For example, $|\{V_s(\pi, b_i, V_{s',i})\}| \leq H$. And $R_{s,a,i}$ is the decomposition of the true reward in the environment by MixNet, which is also bounded, i.e., $|R_{s,a,i}| \leq K$.

Let us define our choice of $\delta_1, \delta_2, \delta_3$ as follows,

$$\begin{aligned}\delta_1(\epsilon) &= \frac{\min\{\epsilon,1\}}{3K(2^{|I|}-1)|A|}\\ \delta_2(\epsilon) &= \frac{\min\{\epsilon,1\}}{3\gamma|S||A|}\\ \delta_3(\epsilon) &= \frac{\min\{\epsilon,1\}}{3\gamma H(2^{|I|}-1)|A|}.\end{aligned} \tag{14}$$

Then,

$$\text{Part 1} \quad \leq \frac{\epsilon}{3}, \text{Part 2} \leq \frac{2\epsilon}{3} \tag{15}$$

Detailed derivations of these two bounds are provided in Supplementary Material I.A. By combining the bounds for Part 1 and Part 2, we have,

$$|V^i_s(\pi_1,\dots) - V^i_s(\pi_2,\dots)| < \frac{\epsilon}{3}+\frac{2\epsilon}{3} = \epsilon. \tag{16}$$

Therefore, the function set $\{V_s(\pi, b_i, V_{s',i})\}$ is equicontinuous. ■

In practice, the term $|V^i_s(\pi_1,\dots) - V^i_s(\pi_2,\dots)|$ represents the distance between two successive value function updates. To reduce abrupt value changes during training, we incorporate the bounds associated with $\delta_1, \delta_2, \delta_3$ as a constraint on the TD loss. The resulting clipped loss is defined as,

$$\begin{aligned}\mathcal{L}^{\text{clip}}(\theta) = \text{clip}\Big(&\mathbb{E}\big[(y - V^i_s(\pi, b_i, V^i_{s'}|\theta))^2\big],\\ &0,\ \min\{\delta_1,\delta_2,\delta_3\}\Big),\end{aligned} \tag{17}$$

where $(y - V^i_s(\pi, b_i, V^i_{s'}|\theta))^2$ is the Time Difference (TD) error $\mathcal{L}_{TD}$ [35]. This clipped objective is used to stabilize the neural approximation of the belief-dependent value function.

Lemma 2: $\phi(\boldsymbol{\pi}^{-i})$ is a convex set, here $\boldsymbol{\pi}^{-i}$ denotes the joint policy of agent other than i.

Proof: The pointwise minimum of an equicontinuous function set is continuous, and $V^i_{\pi,b^*_i}(s) = \max_{b_i} V^i_{\pi,b_i}(s)$ is continuous with respect to all variables. Furthermore, $V^i_{\pi,b^*_i}(s)$ is defined by the discount factor and is bounded. Therefore, the maximum value of $V^i_{\pi,b^*_i}(s)$ exists. ■

Next, in Proposition 2, we proved that the Mutual Awareness Belief Bellman equation updates the Q-function to obtain the optimal Q-value. As a straightforward corollary, the optimal V-value $V^i_{\pi^i_*,\boldsymbol{\pi}^{-i},b_i}(s)$ exists, where π^i_* denotes the optimal policy of agent i. According to the equation $V^i_{\pi^i_*,\boldsymbol{\pi}^{-i},b_i}(s) = max_{\pi^i(a^i|h^i,b_i)}\sum_{a\in\mathcal{A}}\pi^i(a^i|h^i,b_i)[r^i + \sum_{s'\in\mathcal{S}}\mathcal{P}(s'|s,a,\theta)V^i_{\pi^i_*,\boldsymbol{\pi}^{-i},b_i}(s')]$, $\phi(\boldsymbol{\pi}^{-i}) = \phi$ we have $\phi(\boldsymbol{\pi}^{-i}) = \phi$.

Finally, we prove the convexity of $\phi(\boldsymbol{\pi}^{-i})$. Let $\pi^i_{1,*}, \pi^i_{2,*} \in \phi(\boldsymbol{\pi}^{-i})$, where $\boldsymbol{\pi}^{-i}$ is fixed. By the definition of $V^i_{\{\pi^i_*,\boldsymbol{\pi}^{-i}\},b_i}(s)$, for all $s \in \mathcal{S}$, $b \in \Delta(\Theta)$, and $i \in \mathcal{N}$, we have: $V^i_{\{\pi^i_{1,*},\boldsymbol{\pi}^{-i}\},b_i}(s) = V^i_{\{\pi^i_{2,*},\boldsymbol{\pi}^{-i}\},b_i}(s) \geq V^i_{\pi,b_i}(s)$. Therefore, for all $\lambda \in [0,1]$, we also have: $\lambda V^i_{\{\pi^i_{1,*},\boldsymbol{\pi}^{-i}\},b_i}(s) + (1-\lambda)V^i_{\{\pi^i_{2,*},\boldsymbol{\pi}^{-i}\},b_i}(s) \geq V^i_{\pi,b_i}(s)$.

Since $V^i_{\pi,b_i}(s)$ is a mixed Q-value function across multiple agents, it is concave. Thus, we have,

$$\begin{aligned}&V^i_{\{\pi^i_1,\boldsymbol{\pi}^{-i}\},b_i}(s)\\ &= \lambda V^i_{\{\pi^i_1,\boldsymbol{\pi}^{-i}\},b_i}(s) + (1-\lambda)V^i_{\{\pi^i_2,\boldsymbol{\pi}^{-i}\},b_i}(s)\\ &\leq V^i_{\{\lambda\pi^i_1+(1-\lambda)\pi^i_2,\boldsymbol{\pi}^{-i}\},b_i}(s)\end{aligned} \tag{18}$$

By Proposition 1, $V^i_{\pi,b_i}(s)$ is optimal and unique. Therefore, $\lambda\pi^i_{1,*} + (1-\lambda)\pi^i_{2,*} \in \phi(\boldsymbol{\pi}^{-i})$, and $\phi(\boldsymbol{\pi}^{-i})$ is a convex set.

Next, we introduce several theorems and prove that $\phi(x)$ is an upper semicontinuous correspondence.

Theorem 3: Let $\mathcal{T}$ be the Bellman operator defined in Proposition 2. The operator $\mathcal{T}V^i_{\{\pi^i_*,\boldsymbol{\pi}^{-i}\},b_i}(s)$ is continuous with respect to $\boldsymbol{\pi}^{-i}$. The set $\{\mathcal{T}V^i_{\{\pi^i_*,\boldsymbol{\pi}^{-i}\},b_i}(s) \mid V^i_{\{\pi^i_*,\boldsymbol{\pi}^{-i}\},b_i}(s) \text{ is bounded}\}$ is equicontinuous.

Proof: The operator $\mathcal{T}V^i_{\{\pi^i_*,\boldsymbol{\pi}^{-i}\},\theta}(s)$ is continuous, and the set $\{\mathcal{T}V^i_{\{\pi^i_*,\boldsymbol{\pi}^{-i}\},\theta}(s) \mid V^i_{\{\pi^i_*,\boldsymbol{\pi}^{-i}\},\theta}(s) \text{ is bounded}\}$ is equicontinuous. Since $\mathcal{T}V^i_{\{\pi^i_*,\boldsymbol{\pi}^{-i}\},b_i}(s) = \mathcal{T}\mathbb{E}_{p(\theta|H)}[V^i_{\{\pi^i_*,\boldsymbol{\pi}^{-i}\},\theta}(s)]$, the expectation of a continuous function remains continuous, and the expectation over an equicontinuous set remains equicontinuous. This completes the proof. ■

Lemma 3:(Upper Hemicontinuity) [36] Define the optimal value as $v^i = \{v^i_1, v^i_2, \dots, v^i_S\}$. For any convergent sequences where $\pi^i_n \to \pi^i$, $\boldsymbol{\pi}^{-i}_n \to \boldsymbol{\pi}^{-i}$, and $\beta(\boldsymbol{\pi}^{-i}_n) \to v^i$, if it holds that $\pi^i_n \in \phi(\boldsymbol{\pi}^{-i}_n)$ for all n, then the limit satisfies $\pi^i \in \phi(\boldsymbol{\pi}^{-i})$. This closed-graph property implies that the mapping ϕ is an upper hemicontinuous correspondence.

Proof: The detailed proof of this property is established in [36] within the context of Bayesian games. Since our MAB-augmented formulation aligns with their compact action and belief space assumptions, this result directly applies to our operator ϕ ■

We have now proven that ϕ is an upper hemicontinuous correspondence (Lemma 3). It maps $\Delta(A)$ to convex subsets of $\Delta(A)$ (Lemma 2). Since $\phi(\boldsymbol{\pi}^{-i})$ is an upper hemicontinuous correspondence, it is also a closed set for any π. Therefore, the result satisfies the requirements of Kakutani's fixed point theorem, and its equilibrium policy is given by $\phi(\boldsymbol{\pi}^{-i})$.

Proposition 2: Assume that the belief is updated via Bayes' rule, and the spaces of states, actions, and beliefs

are finite. The value function is updated through the belief-Bellman equation, and it converges to the optimal value $Q_*(s^t, \mathbf{a}^t, \{b_i\}_{i=1,2,...,N})$.

Proof: The proof combines the standard convergence proof of the Q value function and Bayesian belief updates, demonstrating that our Q value function forms a contraction mapping. The Banach Fixed-Point Theorem is then applied to complete the proof.

The proof incorporates our Mutual Awareness Belief Bellman equation in the contraction mapping proof of MDP. Based on Bellman equation in Eq. 10, let $\triangle(\Theta)$ be a probability over b, the optimal Q-function of a contraction operator $\mathcal{T}$, defined from a generic function $Q : S \times A \times \triangle(\Theta) \to R$ can be defined as,

$$\begin{aligned}&(\mathcal{T}Q)(s, \mathbf{a}^t, \{b_i\}_{i=1,2,...,N}) = \\ &\max_{\mathbf{a}\in\boldsymbol{\mathcal{A}}}[R(\mathbf{a}^t, \{b_i^t\}_{i=1,2,...,N}) + \gamma \sum P_T(\mathbf{o}^{t+1}|B^t, \mathbf{a}^t) \\ &Q_*(s^{t+1}, \mathbf{a}^{t+1}, \{b_i^{t+1}(s^{t+1})\}_{i=1,2,...,N})].\end{aligned} \tag{19}$$

Next, we prove that $\mathcal{T}$ constitutes a contraction operator such that for any two different Q-functions Q_1 and Q_2 in training process, if $\mathcal{T}Q_1(s, \mathbf{a}_i, \{b_i(s)\}_{i=1,2,...,N}) \geq \mathcal{T}Q_2(s, \mathbf{a}_i, \{b_i(s)\}_{i=1,2,...,N})$, then the following holds,

$$||\mathcal{T}Q_1 - \mathcal{T}Q_2||_\infty \leq \gamma||Q_1 - Q_2||_\infty \tag{20}$$

Specifically, for $\epsilon > 0$ and with fixed $\mathbf{a} \in \boldsymbol{\mathcal{A}}, s \in \mathcal{S}, b \in \triangle(\Theta)$, for Q_1 and Q_2,

$$\begin{aligned}&\max_{\mathbf{a}\in\boldsymbol{\mathcal{A}}}[R(\mathbf{a}^t, \{b_i^t\}_{i=1,2,...,N}) + \\ &\gamma \sum P_T(\mathbf{o}^{t+1}|B^t, \mathbf{a}^t) Q_1(s^{t+1}, \mathbf{a}^{t+1}, \\ &\{b_i^{t+1}(s^{t+1})\}_{i=1,2,...,N})] \\ \geq &\mathcal{T}Q_1 - \epsilon\end{aligned} \tag{21}$$

Note that, in order to ensure the consistency of b'^i, it is necessary to update the belief using Bayes' rule. We also need to fix $\mathbf{o}$, since the computation of the belief and the Q-function depends on $\mathbf{o}$. Therefore, we have,

$$\begin{aligned}&\mathbb{E}[\sum[R(\mathbf{a}^t, \{b_i^t\}_{i=1,2,...,N}) + \\ &\gamma \sum P_T(\mathbf{o}^{t+1}|B^t, \mathbf{a}^t) Q_2(s^{t+1}, \mathbf{a}^{t+1}, \\ &\{b_i^{t+1}(s^{t+1})\}_{i=1,2,...,N})]] \\ \leq &\, max_{\pi(\cdot|h,\theta)} \sum p(\theta|h^i)[R(\mathbf{a}^t, \{b_i^t\}_{i=1,2,...,N}) + \\ &\gamma \sum P_T(\mathbf{o}^{t+1}|B^t, \mathbf{a}^t) Q_2(s^{t+1}, \mathbf{a}^{t+1}, \\ &\{b_i^{t+1}(s^{t+1})\}_{i=1,2,...,N})] + \epsilon\end{aligned} \tag{22}$$

Then,

$$0 \leq \mathcal{T}Q_1 - \mathcal{T}Q_2 \leq \gamma\mathbb{E}||Q_1 - Q_2||_\infty + 2\epsilon. \tag{23}$$

A detailed derivation of the contraction bound is provided in Supplementary Material I.B. Thus, we have,

$$||\mathcal{T}Q_1 - \mathcal{T}Q_2||_\infty \leq \gamma\mathbb{E}||Q_1 - Q_2||_\infty + 2\epsilon \tag{24}$$

Moreover, by definition, since ϵ is arbitrary, we have $||\mathcal{T}Q_1 - \mathcal{T}Q_2||_\infty \leq \gamma\mathbb{E}[||Q_1 - Q_2||_\infty]$.

Finally, since $\mathcal{T}$ is a contraction operator on a Banach space, by the Banach fixed-point theorem, updating $Q(s, \mathbf{a}, \{b_i\}_{i=1,2,...,N})$ using the Bellman operator $\mathcal{T}$ will converge to the optimal value function $Q_*(s, \mathbf{a}, \{b_i\}_{i=1,2,...,N})$. This completes the proof of Proposition 2. ■

B. Progressive Graph Construction

We propose a novel progressive graph construction (PGC) method to obtain MAB, consisting of two components: (i) Agent behaviour update component, which represents the observation transition function in MAB; and (ii) Collaboration relationship update component, which represents the state transition function in MAB.

In PGC, the outputs of the observation transition function and state transition function are represented as $\mathbf{B}_t^o$ and $\mathbf{B}_t^r$. We model all agents in a bi-directional graph, denoted as $G = \{\mathbf{V}, \mathbf{E}\}$. The node set $\mathbf{V} = \{V_1, \ldots, V_N\}, \mathbf{V} = \mathbf{B}_t^o$ represents agents with behavioral features $\mathbf{V} \in \mathbb{R}^{N\times dims}$, reflecting the state and behaviour of each agent. The $dims$ is the dimensionality of the feature encoding. The edge $\mathbf{E} = [E_{i,j}]_{N\times N}$ contains weights between any node pairs V_i and V_j, indicating the cooperative relationship of agents, which satisfies $E_{i,j} \neq E_{j,i}$, $\mathbf{E} \in \mathbb{R}^{N\times N}$. $\mathbf{E}$ is computed through $\mathbf{H} \times \mathbf{W}$, where $\mathbf{H}$ represents a retrieval code for one or more agents with $\mathbf{H} \in \mathbb{R}^{k\times dims}$, and $\mathbf{W}$ is the cooperation-informed edge weights with features $\mathbf{W} \in \mathbb{R}^{N\times dims}$ representing complex multi-level collaborative relationships between agents, $\mathbf{W} = \mathbf{B}_t^r$. Using the retrieval code $\mathbf{H}$, $b_i^t(s^t)$ is constructed to match the distribution of agents based on the edge weights $\mathbf{W}$. Thus, $\mathbf{E} = [b_i^t(s^t)\mathbf{W}^\top]_{i=1,2,\cdots,N}$.

In our method, the neural network of each agent consists of two fully connected layers with a Gated Recurrent Unit (GRU). The last neural layer yields the output, which is the all actions value estimation Q_i^t of agent i in step t. The GRU produces h_i^t, serving as the agent's retrieve vector within the PGC. The PGC module consists of two components, denoted by *B-update* and *RS -Update* as shown in Figure 1. To validate the effectiveness of the method, the transition functions in experiments are implemented by a single fully connected layer, called Update Network. Agent-environment interactions occur over t temporal increments, while agent PGC interactions span l increments, where l equals the product of N (number of agents) and t.

The execution process begins with the **Retrieve**: each agent sends a retrieve embedding h_i^t to the PGC module. The PGC utilizes h_i^t and $\mathbf{B}_t^r$ to calculate $\mathbf{E}_t$. The $\mathbf{E}_t$ is then combined with $\mathbf{B}_t^o$ to derive specific b_i as a response to the retrieve, according to Eq. 25. **Update:** After the decision-making process, the agent transmits z_i^t to PGC, leading to the update of $\mathbf{B}_{t+1}^o$ and $\mathbf{B}_{t+1}^r$, according to Eq. 26 and Eq. 27.

1) Retrieval of Individual MAB: The process of obtaining the MAB involves constructing nodes and edges into a graph. An individual agent retrieves its edge-based belief state b_i^t from the graph, as shown in the upper-left corner of Figure 1. The agent uses its own belief state $b(s^t)$, obtained from the GRU network, to search within the state transition matrix $\mathbf{B}_t^r$ for collaboration values with other agents, which correspond to edge weights E in the graph. The agent then extracts the node features, based on these collaboration values, from the

This article has been accepted for publication in IEEE Transactions on Neural Networks and Learning Systems. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TNNLS.2026.3729623

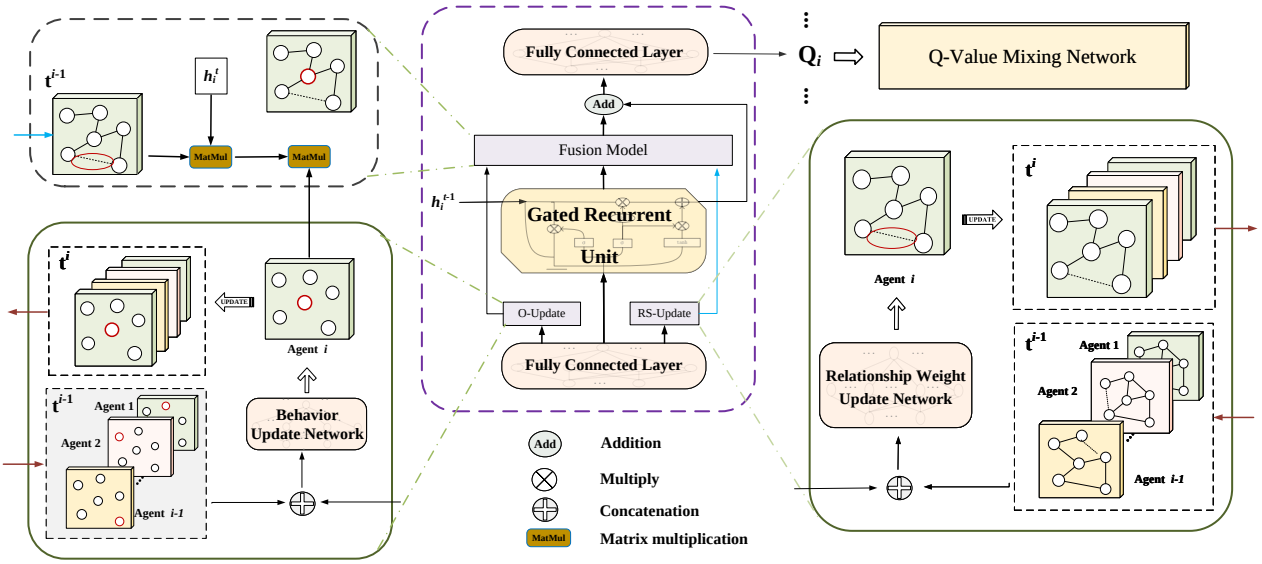

Fig. 1. Algorithm schematic illustrating the interaction of individual agents with PGC during the decision-making process. In the middle of the figure is the policy network of a single agent and the shared PGC module. The policy network of an agent consists of an input layer of a fully connected layer, an intermediate layer of a GRU, and an output layer of a fully connected layer. The PGC module consists of two components, which are denoted in the figure as *B-update* and *RS -Update*, and the solid boxes on the left and right are the internal members of the two components: the update network and transition space. $\mathbf{B}_t^o$ and $\mathbf{B}_t^r$ update processes are also expressed in the solid boxes. The dashed box in the upper left corner shows the process of agent Retrieve. MAB-PGC employs a parameter-sharing approach.

observation probability matrix $\mathbf{B}_t^o$, which represents the node features. These node features are used as sufficient statistics of the observations according to the agent's belief, constructing a belief state $b_i^t(s^t)$ that aligns with the agent's distribution. The process of calculating $b_i^t(s^t)$ is defined as follows,

$$b_i^t(s^t) = \frac{1}{|\mathbf{B}_t^o \times \mathbf{B}_t^{r\top}|} p_i(\theta_i|h_i^t) \times \mathbf{B}_t^{r\top} \times \mathbf{B}_t^o, \qquad (25)$$

where $\mathbf{B}_t^{r\top}$ is the transpose of $\mathbf{B}_t^r$, and $\frac{1}{|\mathbf{B}_t^o \times \mathbf{B}_t^{r\top}|}$ is the specific representation of the normalisation factor η in Eq. 3 in the PGC. Eq. 25 is a specific manifestation of Eq.6, detailing how the MAB is utilized within the PGC module to compute the specific $b_i^t(s^t)$. Finally, $b_i^t(s^t)$ is fed into the last layer of the fully connected neural network of the policy network to obtain an estimate of the value of all actions of the agent $Q_i^t(b_i^t(s^t))$.

Eq. 25 can be viewed as a structured neural approximation of Eq. 3 after applying the agent-specific adjustment in Eq. 6. Therefore, Eq. 25 serves as a structural proxy for this abstract Bayesian update. Specifically, Eq. 3 describes the overall transition process of the joint belief from $B_{t-1}(s^{t-1})$ to $B_t(s^t)$; To ensure computational feasibility, we decouple the belief states into an observation space $\mathbf{B}^o$ and a relationship space $\mathbf{B}^r$. The state transitions of $\mathbf{B}^o$ and $\mathbf{B}^r$ are performed independently via Eq. 26 and Eq. 27. These two update equations replace the integral accumulation process at the neural network level and utilise the hidden embedding h_i^t of GRU to drive spatial updates. In this process, $\mathbf{B}_t^o$ compresses and retains the past temporal history of each agent, serving as an approximate sufficient statistic for individual trajectories; $\mathbf{B}_t^r$ accumulates the historical relational topology via Eq. 27, preserving the history of group interactions among agents. Subsequently, the retrieval mechanism in Eq. 25 recombines the time-shifted $\mathbf{B}_t^o$ and $\mathbf{B}_t^r$. The matrix product $(\mathbf{B}_t^r)^\top \times \mathbf{B}_t^o$, serving as an empirical instantiation of the observed likelihood and historical transition dynamics, mathematically approximates the integral term $P_T(\mathbf{o}^t|\mathbf{a}^{t-1}, s^t)\sum_{s\in\mathcal{S}} P_T(s^t|s^{t-1}, \mathbf{a}^{t-1})B_{t-1}(s^{t-1})$. Based on this, the graph weights $\mathbf{E}_t = p_i(\theta_i|h_i^t) \times (\mathbf{B}_t^r)^\top$ derived from Eq. 6 serve as approximate sufficient statistics for group relationships. Ultimately, driven by the centralised TD error [37], these two sufficient statistics (individual intent h_i^t and relationship weights $\mathbf{E}_t$) are jointly optimised end-to-end to structurally decompose and approximate the intractable true joint Belief (Eq. 3).

2) Update Global MAB: The initial states of $\mathbf{B}_t^o$ and $\mathbf{B}_t^r$ are zero matrices, which are progressively updated as agents interact with the environment. When agent i receives its latest observation, the observation transition function updates $\mathbf{B}_{t,l}^o$ based on this observation and the previous belief space $\mathbf{B}_{t,l-1}^o$. Here, $\mathbf{B}_{t,l}^o$ denotes the belief space after the l-th agent has updated the PGC at time step t, where $l \in \{1, 2, \ldots, N\}$. When $l = N$, $\mathbf{B}_{t,l}^o$ is used as $\mathbf{B}_{t+1}^o$. The same update process applies to $\mathbf{B}_{t,l}^r$. Unlike ACE, the update order is not tied to a fixed agent index. It should be noted that the index l denotes the progressive update order of the internal belief spaces within PGC, rather than a fixed action-execution order among agents. Thus, Eqs. 26 and 27 introduce a representation-level memory update, but do not impose the ACE-style dependency in which one agent must receive the sampled action or predicted state from a prescribed predecessor before making its own decision.

In our implementation, the observation transition function P_T^o and the state transition function P_T^r are parameterized by neural networks with trainable parameters ψ^o and ψ^r,

respectively. As shown in the solid frame on the left side of Figure 1, the $\mathbf{B}_t^o$ matrix of agent features is concatenated with $\mathbf{Z}_i^t$. In this process, $\mathbf{Z}_i^t$ is broadcasted, generating $\mathbf{B'}_t^o \in \mathbb{R}^{N\times(2\cdot dims)}$, which is passed through the Behavior Update Network. $\mathbf{Z}_i^t = \mathbf{1}_N \otimes h_i^t$, where $\otimes$ denotes the Kronecker product [38], vertically broadcasting the individual intent h_i^t across N agents to construct $\mathbf{Z}_i^t$. The expression $\mathbf{B}_{t,l}^o$ is updated by a single agent as follows,

$$\mathbf{B}_{t,l}^o = P_T^o(Cat(\mathbf{B}_{t,l-1}^o, \mathbf{Z}_i^t)|\mathbf{a}^{t,l-1}, s^{t,l-1}; \psi^o) \cdot \mathbf{B}_{t,l-1}^o, \quad (26)$$

where $\mathbf{a}^{t,l-1}$ represents joint actions taken by $l-1$ agents who have made decisions at step t, and $s^{t,l-1}$ indicates the state after these $l-1$ agents have taken actions. $Cat(\cdot)$ means concatenation.

Whenever an agent is about to execute an action, the transition function computes $\mathbf{B}_{t,l}^r$ by combining the action with $\mathbf{B}_t^r$. As shown in the solid frame on the right side of Figure 1, the $\mathbf{B}_t^r$ matrix of agent features is concatenated with $\mathbf{Z}_i^t$. In this process, $\mathbf{Z}_i^t$ is broadcast to generate $\mathbf{B}_t^{r\prime} \in \mathbb{R}^{N\times(2\cdot dims)}$, which is passed through the Relationship Weight Update Network, so as to update an agent's collaboration with other agents in real-time. The expression $\mathbf{B}_{t,l}^r$ is updated by a single agent as follows,

$$\mathbf{B}_{t,l}^r = P_T^r(Cat(\mathbf{B}_{t,l-1}^r, \mathbf{Z}_i^t)|s^{t,l-1}, \mathbf{a}^{t,l-1}; \psi^r) \cdot \mathbf{B}_{t,l-1}^r. \quad (27)$$

Here, $\mathbf{Z}_i^t$ is broadcast as a temporal proxy embedding extracted from the GRU. By encoding historical observation-action trajectories, $\mathbf{Z}_i^t$ serves as a representation of agent i's current policy intention. Subsequently, the operation $Cat(\mathbf{B}_{t,l-1}^r, \mathbf{Z}_i^t)$ is executed. By injecting these individual policy intentions into the shared relational space, this operation ensures that every agent node in the graph constructed by PGC can dynamically adjust the MAB transition probability P_T^r based on these individual behavioural intentions. Since the updates involve $\mathbf{B}_t^r$ and $\mathbf{B}_t^o$, which represent all agent nodes, l is independent of the decision sequence. In ACE, however, agent i must explicitly pass its sampled action and state to the next agent $i+1$, thereby creating strict sequential execution blocks at the decision level. Furthermore, Eqs. 25, 26 and 26 ensure that PGC supports interaction with any individual agent without requiring information from all agents prior to execution; consequently, we refer to MAB-PGC as a weakly centralized information center method. Although MAB-PGC enables direct decision-making by individual agents, a shared component remains.

Remark 1*:* Fundamentally different from existing step-wise methods that discard graph structures between steps, the progressive update mechanism in Eq. 27 maintains a persistent relational graph. By iteratively integrating new localized features $\mathbf{Z}_i^t$ into the historical relational space $\mathbf{B}_{t-1}^r$, PGC acts as a temporal buffer. When the local observation of an agent is temporarily occluded or noisy, the persistent edges prevent the immediate severing of collaborative links. This persistent memory effectively reduces collaborative fragmentation, ensuring that agents can trust stable topological connections to execute long-term coordination policies.

The updates of $\mathbf{B}^o$ and $\mathbf{B}^r$ in Eqs. 26 and 27 are forward computations that refresh the belief representation. The resulting MAB $b_i^t(s^t)$ is then used as the input to the individual utility $Q_i(b_i^t(s^t), a_i^t)$, and therefore affects the TD target and the value-based policy improvement. During training, the PGC parameters are optimized by the TD loss together with the value network parameters, so that the learned belief representation supports value estimation. Thus, PGC influences policy learning through the MAB-dependent Q-value, while the policy update itself remains governed by the value-based Bellman objective (Proposition 2). In a team reward setting, our method adopts the training strategy of QMIX, where a MIXNet is responsible for reward allocation, enabling each agent to obtain the effectiveness of the MAB distribution provided by the PGC. The update mechanism demonstrates that each update can incorporate information from a single agent. This allows a single agent to perform updates for all agent nodes, enabling the dynamic adjustment of each node's features based on changes in agent behavior. This approach effectively addresses the issue of information lag. During the transition from step t to $t+1$, each agent integrates its real-time information, mitigating the distortion problem observed in the ACE [18] method. Additionally, it avoids the need for a strict ordering of state updates.

The complete algorithmic pseudocode of MAB-PGC is detailed in Supplementary Material I.D To verify its computational viability, a comprehensive complexity analysis is also provided therein. This analysis theoretically proves that our broadcast mechanism achieves scale-invariance regarding the agent number and significantly bounds the relative overhead (e.g., from 201.56% down to 26.56% in standard scenarios).

IV. Experiments

In this section, we validate the effectiveness of the proposed approach by comparing it with several strong baselines against diverse tests. We first introduce the two primary environments selected for experimentation and explain our rationale for choosing them, while also outlining the settings employed during algorithm training to ensure fairness in comparing algorithms. Subsequently, we analyze the results of each comparison algorithm and MAB-PGC across these two main environments. Finally, to analyze the impact of information lag on team collaboration, we present an ablation study on the PGC update frequency in this section. In addition, the ablation study on joint belief state construction and the validation of our approach's generalizability in random environments using StarCraft Multi-Agent Challenge v2 (SMACv2) [39] are provided in Supplementary Material II.D and II.E

A. Environments and Training Settings

For the experimental evaluation, we conducted tests on two different types of environments, including monotonic and non-monotonic. We chose StarCraft Multi-Agent Challenge (SMAC) [40] as the monotonic environment for algorithm validation because the optimal action of each agent does not depend on the actions of other agents [41]. SMAC is an environment for research in the field of cooperative-MARL

This article has been accepted for publication in IEEE Transactions on Neural Networks and Learning Systems. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TNNLS.2026.3729623

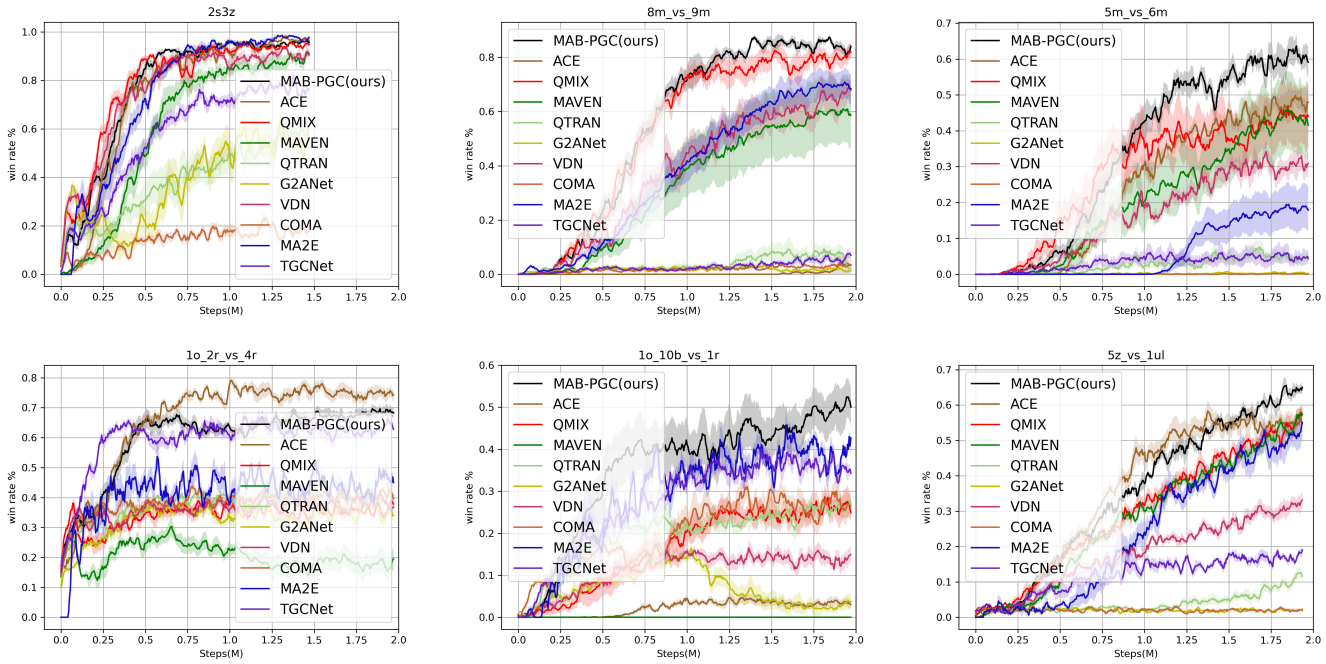

Fig. 2. Comparison of MAB-PGC against baselines on 6 SMAC maps.

TABLE I
WIN RATES AND VARIANCES OF DIFFERENT ALGORITHMS ACROSS MAPS

Algorithm	8m_vs_9m	5m_vs_6m	1o_2r_vs_4r	1o_10b_vs_1r	5z_vs_1ul	2s3z	MMM	2c_vs_64zg	MMM2
MAB-PGC(ours)	**0.850** ± 0.005	**0.619** ± 0.024	0.644 ± 0.010	**0.488** ± 0.027	**0.619** ± 0.005	**0.944** ± 0.002	**0.994** ± 0.000	**0.938** ± 0.000	**0.588** ± 0.115
ACE	0.038 ± 0.010	0.394 ± 0.011	**0.788** ± 0.005	0.031 ± 0.001	0.519 ± 0.014	**0.969** ± 0.000	-	-	-
QMIX	0.788 ± 0.009	0.400 ± 0.060	0.369 ± 0.005	0.269 ± 0.005	0.538 ± 0.013	**0.956** ± 0.003	-	-	-
MAVEN	0.588 ± 0.121	0.350 ± 0.079	0.194 ± 0.016	0.000 ± 0.000	0.531 ± 0.014	0.900 ± 0.005	-	-	-
QTRAN	0.025 ± 0.000	0.069 ± 0.003	0.431 ± 0.006	0.244 ± 0.013	0.113 ± 0.006	0.613 ± 0.029	-	-	-
G2ANet	0.013 ± 0.000	0.000 ± 0.000	0.388 ± 0.014	0.044 ± 0.005	0.006 ± 0.000	0.519 ± 0.034	-	-	-
VDN	0.669 ± 0.048	0.300 ± 0.017	0.363 ± 0.007	0.156 ± 0.010	0.381 ± 0.009	0.894 ± 0.021	-	-	-
COMA	0.044 ± 0.002	0.000 ± 0.000	0.444 ± 0.005	0.244 ± 0.005	0.013 ± 0.000	0.175 ± 0.017	-	-	-
MA2E	0.681 ± 0.055	0.169 ± 0.046	0.456 ± 0.003	**0.500** ± 0.011	0.500 ± 0.037	**0.938** ± 0.005	**0.994** ± 0.000	**0.950** ± 0.001	0.438 ± 0.087
TGCNet	0.069 ± 0.004	0.031 ± 0.000	0.625 ± 0.005	0.319 ± 0.013	0.144 ± 0.003	0.800 ± 0.015	0.750 ± 0.041	0.494 ± 0.020	0.406 ± 0.143

based on Blizzard's StarCraft II game. SMAC concentrates on decentralised micromanagement scenarios, where each unit of the game is controlled by an individual RL agent [40]. The experiments were carried out using nine distinct maps: $2s3z$, $5m\_vs\_6m$, $8m\_vs\_9m$ [40], $1o\_2r\_vs\_4r$, $1o\_10b\_vs\_1r$, $5z\_vs\_1ul$, MMM,$MMM2$,$2c\_vs\_64zg$ [42].

We selected the *Pursuit* environment [43] as our non-monotonic environment for the experiments. In this environment, the actions of a single agent alone do not yield rewards. Instead, collaboration with other agents is necessary to obtain a reward. The choice of Pursuit as the experimental environment was motivated by its requirement for agents to coordinate their actions effectively in order to achieve rewards. This necessitates the ability of algorithms to accurately perceive the actions of other agents and the global state, as well as determine the true reward potential of their own actions. In the environment settings, the contrastive learning method controls a group of 10 pursuer agents tasked with capturing 50 evader agents. The map size is 16×16 and the observable range of the agent is 5×5, with a tag reward of 0.01, a catch reward of 5, and an urgency reward of -0.1.

The global state is used in the training process to optimize the estimation network of Q_{tol}, but not in the decision-making process of the agents. The agents can only obtain observations. We ran all the experiments eight times with different random seeds. We set the discount factor as 0.99 and use the RMSprop [44] optimizer with a learning rate of $5e^{-4}$. The ϵ-greedy is used for exploration with ϵ annealed linearly from 1.0 to 0.05 in 0.5 M steps. The batch size is 32, updating the target networks every 200 episodes. Hyperparameter settings are shown in Table I in Supplementary Material II.

B. Performance on SMAC Scenarios

The experimental results in the SMAC environment are shown in Figure 2. Table I shows the average win rates and variances of different algorithms across nine test scenarios at the latest test. The results indicate that MAB-PGC achieves the best or competitive win rates on most tasks, showing clear advantages over mainstream baselines such as QMIX and competitive performance against recent methods such as MA2E [45]. Value decomposition methods, including VDN, QMIX, and MAVEN, satisfy the IGM condition but fail to

This article has been accepted for publication in IEEE Transactions on Neural Networks and Learning Systems. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TNNLS.2026.3729623

guarantee accurate estimates of individual agents' rewards. As a result, these methods often struggle to achieve higher rewards or win rates, as some agents receive incorrect reward estimates for their actions.

The ACE method proposed for SE-MDP demonstrates certain advantages on the $1o\_2r\_vs\_4r$ map, achieving the highest win rate of 78.8%. This is attributed to the small number of agents in this map, which mitigates the severity of state distortion caused by state transitions of SE-MDP. As a result, ACE performs better on less complex maps with a moderate number of agents. However, ACE is less effective on maps with larger numbers of agents, achieving only a 3.8% and 3.1% win rate in $8m\_vs\_9m$ and $1o\_10b\_vs\_1r$. This poor performance arises from increased state distortion as the number of agents and the length of the state sequence in SE-MDP grow.

In contrast, our method performs consistently well across the tested maps. By addressing credit assignment without introducing the state distortion seen in sequential approaches, MAB-PGC achieves strong win rates on maps with large numbers of agents and complex configurations. Notably, on the $1o\_10b\_vs\_1r$ map, which features a small maze, the controlled agents must navigate obstacles to launch effective attacks. Additionally, since the bane units perish after dealing damage while the enemy's life gradually regenerates, synchronized attacks by the bane units are crucial for success. Value decomposition algorithms struggle to coordinate such attacks. This is because standard credit assignment mechanisms often fail to properly credit an early agent's action whose true value depends on a temporally distant, coordinated action by a teammate. Our approach achieved a competitive win rate of 48.8%.

Another prominent paradigm is graph-based communication and coordination. G2ANet is a classic graph-attention method, but it yields 1.3% and 0.0% win rates on $8m\_vs\_9m$ and $5m\_vs\_6m$. TGCNet is a more recent graph-based approach. On maps that include a dedicated observer unit with a wide field of view, such as $1o\_2r\_vs\_4r$ and $1o\_10b\_vs\_1r$, TGCNet achieves competitive performance with win rates of 62.5% and 31.9%, close to MAB-PGC's 64.4% and 48.8%. We attribute this to its effective use of a Transformer-based decoder [21], which distributes the large-scale observational information to the team. In fact, TGCNet assumes that the global state s cannot be obtained during training under the CTDE framework. To ensure a fair comparison, we replace the MIXNet used by TGCNet to approximate the global state s with a standard QMIXNet [14] that can directly utilize s. Concurrently, the individual agents in TGCNet still retain their original communication graph during the decision-making phase, which establishes a rigorous baseline to directly compare the structural effectiveness of their graph topology against our proposed Progressive Graph Construction module. However, on maps where agents rely on limited local observations, TGCNet achieves 80.0% on the $2s3z$ map, but drops to 6.9%, 3.1%, and 14.4% on the $8m\_vs\_9m$, $5m\_vs\_6m$, and $5z\_vs\_1ul$ maps. Both G2ANet and TGCNet share a structural characteristic: they construct a communication graph at each step based entirely on current step information. This step-wise construction may limit the capacity to capture long-term collaborative dependencies. In contrast, our MAB-PGC explicitly maintains a history of collaborative relationships through progressive graph construction. This design allows the method to integrate historical context into team dynamics, achieving win rates of 94.4%, 85.0%, 61.9%, and 61.9% on $2s3z$, $8m\_vs\_9m$, $5m\_vs\_6m$, and $5z\_vs\_1ul$, respectively. Conceptually similar to MAB-PGC, MA2E is a recent method designed as a plug-and-play module to enhance baseline algorithms. MA2E excels in specific coordination tasks. For instance, on the $1o\_10b\_vs\_1r$ map, where controlled bane units must navigate a maze and perform strictly synchronized suicide attacks while the enemy regenerates health, standard value decomposition algorithms fail to assign credit to temporally distant, highly coordinated actions. In this scenario, both MA2E and MAB-PGC perform effectively, achieving win rates of 50.0% and 48.8%, respectively. However, the performance of MA2E varies across different environments. In dense scenarios like $5m\_vs\_6m$, the win rate of MA2E is 16.9%, whereas MAB-PGC maintains a win rate of 61.9%.

C. Ablation Study on PGC Update frequency

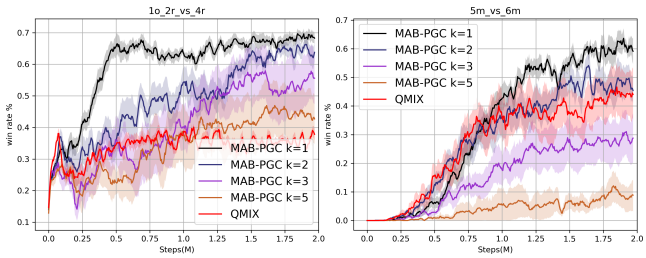

Fig. 3. Ablation study on the impact of information lag controlled by the PGC update frequency k. Standard MAB-PGC uses $k = 1$, whereas $k \in \{2, 3, 5\}$ simulates increasing degrees of information lag. The performance is evaluated in $5m\_vs\_6m$ and $1o\_2r\_vs\_4r$.

To further explore and verify the negative impact of information lag on team collaboration, we conducted an ablation study on the frequency parameter k of the agent's PGC updates. The variable k specifies that the agent is only permitted to update the information into the PGC once every k time steps. In the standard MAB-PGC method, $k = 1$. In this ablation experiment, we additionally evaluated three increasing delay frequencies: $k = 2$, $k = 3$ and $k = 5$.

It should be noted that the setting of $k = 2$ logically approximates the information lag bottleneck found in traditional multi-agent communication methods. In traditional methods, information is typically broadcast statically once at the start of a time step; as some agents adjust their policies based on this information, the initial broadcast becomes out of step with the actual dynamic execution state. When $k = 2$ is set, the PGC ceases to update and retains the previous information for one time step, thereby replicating this phenomenon of information lag caused by asynchronous internal decision-making. Meanwhile, $k = 3$ and $k = 5$ further amplify the severity of this information lag.

To compare the impact of information lag under different observation conditions, we conducted experiments on

two maps in the SMAC environment: $5m\_vs\_6m$ (without wide-range observers) and $1o\_2r\_vs\_4r$ (with wide-range observers). The results are shown in Fig. 3. The experimental results indicate that the performance of the method is naturally inversely proportional to the degree of information lag. In the scenario without wide-range observers ($5m\_vs\_6m$), when $k = 2$, although information lag similar to that of traditional methods was introduced, the collaborative performance remained superior to the baseline due to the advantages of MAB. Nevertheless, when the delay was further increased ($k = 3$ and $k = 5$), severe information gaps led to a sharp decline in team performance. In scenarios with wide-range observers ($1o\_2r\_vs\_4r$), all MAB-PGC variants with delay settings ($k = 2, 3, 5$) consistently outperformed the baseline. This is because wide-range observers possess a broader field of view, which effectively mitigates the negative impact of reduced PGC update frequencies.

Extended evaluations in the Pursuit environment, including qualitative and quantitative analyses of coordination dynamics, are provided in Supplementary Material II.A, II.B and II.C. In our experiments, we observed an emergent grouping and temporal coordination phenomenon. The relational graph is updated progressively based on PGC, ensuring a smooth evolution of relationships and avoiding the instantaneous mutations that inherently cause severe collaborative fragmentation. To better validate these qualitative observations and evaluate this collaborative phenomenon, we introduce two new metrics in the supplementary material, the Subgroup Stability Index (SSI) and the Attention Consistency Score (ACS). These metrics demonstrate that MAB-PGC promotes sustained subgroup affiliation and maintains a stable attention mechanism.

V. Conclusion

In this paper, we propose MAB-PGC, a novel method designed to enhance decentralized coordination in multi-agent systems. By mathematically extending the individual belief state to incorporate the MAB, which captures joint historical observations and actions, our method equips agents with a more precise and comprehensive understanding of both the global state and collective intents. Furthermore, the progressive update mechanism within the PGC module effectively mitigates information lag and avoids message distortion, all while strictly preserving the conditional independence required for decentralized execution. Comprehensive empirical evaluations across both monotonic and non-monotonic environments demonstrate that MAB-PGC achieves strong and competitive performance against existing baselines, validating its generalizability, improved collaborative efficiency, and robustness. Our future research will focus on two primary directions to further extend this framework. First, we target real-world physical deployments. We aim to transition MAB-PGC from simulated environments into specific intelligent manufacturing applications involving robotic arms. Second, we will enhance the underlying algorithmic topology. By upgrading the current simple graph structure to higher-order representations like hypergraphs, the PGC module will capture intricate team-wide dependencies. This generates an even more expressive MAB. Ultimately, such structural advancements will further elevate cooperative performance in complex interaction scenarios.

Acknowledgment

The authors express their sincere gratitude to editors and reviewers for their insightful and constructive comments on this paper. During the preparation of this paper, we used generative AI tools to improve the readability of the paper only. The ideas, methodology, and experimental results presented in this paper are original.

References

[1] Y. Zhang, M. Yue, J. Wang, and S. Yoo, "Multi-agent graph-attention deep reinforcement learning for post-contingency grid emergency voltage control," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 35, no. 3, pp. 3340–3350, 2024.

[2] S. Li, R. Xu, J. Xiu, Y. Zheng, P. Feng, Y. Ma, B. An, Y. Yang, and X. Liu, "Robust multi-agent reinforcement learning by mutual information regularization," *IEEE Transactions on Neural Networks and Learning Systems*, 2025.

[3] M. Zhou, J. Luo, J. Villella, Y. Yang, D. Rusu, J. Miao, W. Zhang, M. Alban, I. Fadakar, Z. Chen *et al.*, "Smarts: Scalable multi-agent reinforcement learning training school for autonomous driving," *arXiv preprint arXiv:2010.09776*, 2020.

[4] M. Hüttenrauch, A. Šošić, and G. Neumann, "Deep reinforcement learning for swarm systems," *Journal of Machine Learning Research*, vol. 20, no. 54, pp. 1–31, 2019.

[5] Y. Yu, J. Guo, C. K. Ahn, and Z. Xiang, "Neural adaptive distributed formation control of nonlinear multi-uavs with unmodeled dynamics," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 34, no. 11, pp. 9555–9561, 2023.

[6] K. Shao, Z. Tang, Y. Zhu, N. Li, and D. Zhao, "A survey of deep reinforcement learning in video games," *arXiv preprint arXiv:1912.10944*, 2019.

[7] X. Xu, R. Li, Z. Zhao, and H. Zhang, "Stigmergic independent reinforcement learning for multiagent collaboration," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 33, no. 9, pp. 4285–4299, 2022.

[8] A. Tampuu, T. Matiisen, D. Kodelja, I. Kuzovkin, K. Korjus, J. Aru, J. Aru, and R. Vicente, "Multiagent cooperation and competition with deep reinforcement learning," *PloS one*, vol. 12, no. 4, p. e0172395, 2017.

[9] R. Lu, Y. Zhu, D. Zhao, Y. Liu, and Y. He, "Last-iterate convergence to approximate nash equilibria in multiplayer imperfect information games," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 36, no. 8, pp. 13 859–13 873, 2025.

[10] J. Chen, J. Chen, T. Lan, and V. Aggarwal, "Multi-agent covering option discovery based on kronecker product of factor graphs," *IEEE Transactions on Artificial Intelligence*, 2022.

[11] Y. Li, J. Li, and J. Pang, "A graph attention mechanism-based multiagent reinforcement-learning method for task scheduling in edge computing," *Electronics*, vol. 11, no. 9, p. 1357, 2022.

[12] J. Foerster, G. Farquhar, T. Afouras, N. Nardelli, and S. Whiteson, "Counterfactual multi-agent policy gradients," in *Proceedings of the AAAI conference on artificial intelligence*, 2018.

[13] P. Sunehag, G. Lever, A. Gruslys, W. M. Czarnecki, V. Zambaldi, M. Jaderberg, M. Lanctot, N. Sonnerat, J. Z. Leibo, K. Tuyls *et al.*, "Value-decomposition networks for cooperative multi-agent learning," *arXiv preprint arXiv:1706.05296*, 2017.

[14] T. Rashid, M. Samvelyan, C. S. De Witt, G. Farquhar, J. Foerster, and S. Whiteson, "Monotonic value function factorisation for deep multi-agent reinforcement learning," *The Journal of Machine Learning Research*, vol. 21, no. 1, pp. 7234–7284, 2020.

[15] K. Son, D. Kim, W. J. Kang, D. E. Hostallero, and Y. Yi, "Qtran: Learning to factorize with transformation for cooperative multi-agent reinforcement learning," in *International conference on machine learning*. PMLR, 2019, pp. 5887–5896.

[16] Y. Hou, M. Sun, Y. Zeng, Y.-S. Ong, Y. Jin, H. Ge, and Q. Zhang, "A multi-agent cooperative learning system with evolution of social roles," *IEEE transactions on evolutionary computation*, 2023.

[17] Y. Jin, S. Wei, J. Yuan, and X. Zhang, "Hierarchical and stable multiagent reinforcement learning for cooperative navigation control," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 34, no. 1, pp. 90–103, 2023.

[18] C. Li, J. Liu, Y. Zhang, Y. Wei, Y. Niu, Y. Yang, Y. Liu, and W. Ouyang, "Ace: Cooperative multi-agent q-learning with bidirectional action-dependency," in *Proceedings of the AAAI conference on artificial intelligence*, 2023, pp. 8536–8544.

[19] T. He, Y. Liu, Y.-S. Ong, X. Wu, and X. Luo, "Polarized message-passing in graph neural networks," *Artificial Intelligence*, vol. 331, p. 104129, 2024.

[20] Y. Liu, W. Wang, Y. Hu, J. Hao, X. Chen, and Y. Gao, "Multi-agent game abstraction via graph attention neural network," in *Proceedings of the AAAI Conference on Artificial Intelligence*, 2020, pp. 7211–7218.

[21] Z. Zhang, B. He, B. Cheng, and G. Li, "Bridging training and execution via dynamic directed graph-based communication in cooperative multi-agent systems," in *Proceedings of the AAAI Conference on Artificial Intelligence*, Philadelphia, Pennsylvania, USA, 2025.

[22] Y. Liang, H. Wu, and H. Wang, "Asm-ppo: Asynchronous and scalable multi-agent ppo for cooperative charging," in *Proceedings of the 21st International Conference on Autonomous Agents and Multiagent Systems*, 2022, pp. 798–806.

[23] F. A. Oliehoek, C. Amato *et al.*, *A concise introduction to decentralized POMDPs*. Springer, 2016, vol. 1.

[24] R. Nair, M. Tambe, M. Yokoo, D. Pynadath, and S. Marsella, "Taming decentralized pomdps: Towards efficient policy computation for multiagent settings," in *IJCAI*, vol. 3, 2003, pp. 705–711.

[25] G. Papoudakis, F. Christianos, A. Rahman, and S. V. Albrecht, "Dealing with non-stationarity in multi-agent deep reinforcement learning," *arXiv preprint arXiv:1906.04737*, 2019.

[26] A. S. M. Tayeen, M. Biswal, A. Mtibaa, and S. Misra, "Analysis of independent learning in network agents: A packet forwarding use case," *arXiv preprint arXiv:2202.02349*, 2022.

[27] C. Zhu, M. Dastani, and S. Wang, "A survey of multi-agent reinforcement learning with communication," *arXiv preprint arXiv:2203.08975*, 2022.

[28] S. Sukhbaatar, R. Fergus *et al.*, "Learning multiagent communication with backpropagation," *Advances in neural information processing systems*, vol. 29, 2016.

[29] Z. Pu, H. Wang, Z. Liu, J. Yi, and S. Wu, "Attention enhanced reinforcement learning for multi agent cooperation," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 34, no. 11, pp. 8235–8249, 2023.

[30] M. L. Littman, "Markov games as a framework for multi-agent reinforcement learning," in *Machine learning proceedings 1994*. Elsevier, 1994, pp. 157–163.

[31] J. K. Gupta, M. Egorov, and M. Kochenderfer, "Cooperative multi-agent control using deep reinforcement learning," in *International conference on autonomous agents and multiagent systems*, 2017, pp. 66–83.

[32] J. K. Terry, N. Grammel, S. Son, B. Black, and A. Agrawal, "Revisiting parameter sharing in multi-agent deep reinforcement learning," *arXiv preprint arXiv:2005.13625*, 2020.

[33] C. Yu, A. Velu, E. Vinitsky, J. Gao, Y. Wang, A. Bayen, and Y. Wu, "The surprising effectiveness of ppo in cooperative multi-agent games," *Advances in neural information processing systems*, pp. 24 611–24 624, 2022.

[34] P. Feng, R. Shi, S. Wang, Q. Wu, X. Yu, and W. Wu, "Lyapunov-informed multi-agent reinforcement learning for multi-robot cooperation tasks," *IEEE Transactions on Automation Science and Engineering*, 2025.

[35] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski *et al.*, "Human-level control through deep reinforcement learning," *nature*, vol. 518, no. 7540, pp. 529–533, 2015.

[36] S. Li, J. Guo, J. Xiu, R. Xu, X. Yu, J. Wang, A. Liu, Y. Yang, and X. Liu, "Byzantine robust cooperative multi-agent reinforcement learning as a bayesian game," *arXiv preprint arXiv:2305.12872*, 2023.

[37] L. Kong, J. Feng, H. Liu, D. Tao, Y. Chen, and M. Zhang, "Mag-gnn: Reinforcement learning boosted graph neural network," *Advances in Neural Information Processing Systems*, pp. 12 000–12 021, 2023.

[38] A. Graham, *Kronecker products and matrix calculus with applications*. Courier Dover Publications, 2018.

[39] B. Ellis, J. Cook, S. Moalla, M. Samvelyan, M. Sun, A. Mahajan, J. Foerster, and S. Whiteson, "Smacv2: An improved benchmark for cooperative multi-agent reinforcement learning," *Advances in Neural Information Processing Systems*, pp. 37 567–37 593, 2023.

[40] M. Samvelyan, T. Rashid, C. S. De Witt, G. Farquhar, N. Nardelli, T. G. Rudner, C.-M. Hung, P. H. Torr, J. Foerster, and S. Whiteson, "The starcraft multi-agent challenge," *arXiv preprint arXiv:1902.04043*, 2019.

[41] T. Gupta, A. Mahajan, B. Peng, W. Böhmer, and S. Whiteson, "Uneven: Universal value exploration for multi-agent reinforcement learning," in *International Conference on Machine Learning*. PMLR, 2021, pp. 3930–3941.

[42] T. Wang, J. Wang, C. Zheng, and C. Zhang, "Learning nearly decomposable value functions via communication minimization," *arXiv preprint arXiv:1910.05366*, 2019.

[43] J. Terry, B. Black, N. Grammel, M. Jayakumar, A. Hari, R. Sullivan, L. S. Santos, C. Dieffendahl, C. Horsch, R. Perez-Vicente *et al.*, "Pettingzoo: Gym for multi-agent reinforcement learning," *Advances in Neural Information Processing Systems*, pp. 15 032–15 043, 2021.

[44] J. Duchi, E. Hazan, and Y. Singer, "Adaptive subgradient methods for online learning and stochastic optimization." *Journal of machine learning research*, vol. 12, no. 7, 2011.

[45] S. Kang, Y. Lee, G. Kim, S. Chong, and S.-Y. Yun, "MA^{2}E: Addressing partial observability in multi-agent reinforcement learning with masked auto-encoder," in *The Thirteenth International Conference on Learning Representations*, 2025.